Homework 4: Bayesian Inference for Mixture Models

BIOS 731 – Advanced Statistical Computing

GitHub Link

Context and learning objectives

This assignment reinforces ideas in the **Bayesian Computing** and **Cluster Computing** modules. We focus specifically on implementing a Bayesian K-Mixture Model using Gibb's sampling and variational inference.

Due Date and Submission

Submit (via Canvas) a PDF knitted from a `.Rmd` or `.qmd` file. Your PDF should include the web address of the GitHub repository containing your work for this assignment. Due date is Wednesday, 4/8 at 10:00AM.

Point distribution

Problem	Points
Problem 0	10
Problem 1	20
Problem 2	20
Problem 3	20
Problem 4	30

Problem 0

This "problem" focuses on the structure of your submission, especially the **use of git and GitHub** for reproducibility, **R Projects** to organize your work, **Quarto/R Markdown** to write reproducible reports, **relative paths** to load local files, and reasonable naming conventions for your files.

To that end:

- Create a public GitHub repository and a local R Project. I suggest naming the repo/directory `bios731_hw3_YourLastName` (e.g., `bios731_hw3_wrobel`).
- Push your **entire project folder** to GitHub.
- Submit a PDF knitted from your `.Rmd`/`.qmd` file to Canvas.
  - Your solutions should be implemented in your `.Rmd`/`.qmd` file.
  - Your git commit history should reflect your workflow (i.e., multiple meaningful commits; avoid a single "final" commit with all work).

Bayesian K-Mixture Model

We consider a Bayesian Mixture Model of unit-variance univariate Gaussians. There are:

- K mixture components, corresponding to K Gaussian distributions with means $= \{\mu_1, \dots, \mu_K\}$
- cluster assignments $c_i \in \{1, \dots, K\}$
- observations $y_i \in \{1, \dots, n\}$

Data generating model is

$$
\begin{aligned}
\mu &\sim N(0, \sigma^2) \\
c_i &\sim Categorical(1/K. \dots, 1/K) \\
y_i | c_i, \mu &\sim N(c_i^T \mu, 1)
\end{aligned}
$$

Problem 1: Gibbs sampler derivations and implementation

Derive a Gibb's sampler for the posterior distribution, $p(\mu, \mathbf{c}|y)$. Show the derivations and resulting Gibb's sampler updates. Specifically, derive:

- $p(c_i|y_i, \mu)$
- $p(\mu_k|y, c)$

Implement the Gibb's sampler in R. Your Gibb's R function should

- run the sampler
- store posterior draws
- return posterior summaries (estimates of the cluster assignments and means).

1. Derivation of $p(c_i \mid y_i, \mu)$

We first derive the conditional posterior distribution of the cluster assignment c_i.

By Bayes' theorem,

$$p(c_i = k \mid y_i, \mu) \propto p(c_i = k)\, p(y_i \mid c_i = k, \mu)$$

since the normalizing constant does not depend on k. ### Prior for c_i

Because

$$c_i \sim \text{Categorical}\left(\frac{1}{K}, \dots, \frac{1}{K}\right),$$

we have

$$p(c_i = k) = \frac{1}{K}.$$

Likelihood for y_i

From the model,

$$y_i \mid c_i = k, \mu \sim N(\mu_k, 1),$$

so the likelihood is

$$p(y_i \mid c_i = k, \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right).$$

Combine prior and likelihood

Thus,

$$p(c_i = k \mid y_i, \mu) \propto \frac{1}{K} \cdot \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right).$$

Since $\frac{1}{K}$ and $\frac{1}{\sqrt{2\pi}}$ do not depend on k, they can be absorbed into the proportionality constant:

$$p(c_i = k \mid y_i, \mu) \propto \exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right).$$

Normalize

To obtain a valid categorical distribution, normalize over all K components:

$$p(c_i = k \mid y_i, \mu) = \frac{\exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right)}{\sum_{h=1}^{K} \exp\left(-\frac{1}{2}(y_i - \mu_h)^2\right)}.$$

Therefore,

$$c_i \mid y_i, \mu \sim \text{Categorical}(\pi_{i1}, \dots, \pi_{iK})$$

where

$$\pi_{ik} = \frac{\exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right)}{\sum_{h=1}^{K} \exp\left(-\frac{1}{2}(y_i - \mu_h)^2\right)}.$$

2. Derivation of $p(\mu_k \mid y, c)$

Next, we derive the conditional posterior for the component mean μ_k.

By Bayes' theorem,

$$p(\mu_k \mid y, c) \propto p(\mu_k) \prod_{i:c_i=k} p(y_i \mid \mu_k).$$

Only the observations assigned to cluster k contribute to this conditional distribution.

Prior for μ_k

From the prior,

$$\mu_k \sim N(0, \sigma^2),$$

so

$$p(\mu_k) \propto \exp\left(-\frac{\mu_k^2}{2\sigma^2}\right).$$

Likelihood for observations assigned to cluster k

For all observations with $c_i = k$,

$$y_i \mid \mu_k \sim N(\mu_k, 1),$$

so

$$p(y_i \mid \mu_k) \propto \exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right).$$

Thus,

$$\prod_{i:c_i=k} p(y_i \mid \mu_k) \propto \exp\left(-\frac{1}{2}\sum_{i:c_i=k}(y_i - \mu_k)^2\right).$$

Combine prior and likelihood

Therefore,

$$p(\mu_k \mid y, c) \propto \exp\left(-\frac{\mu_k^2}{2\sigma^2}\right)\exp\left(-\frac{1}{2}\sum_{i:c_i=k}(y_i - \mu_k)^2\right).$$

Combining the exponents:

$$p(\mu_k \mid y, c) \propto \exp\left[-\frac{1}{2}\left(\frac{\mu_k^2}{\sigma^2} + \sum_{i:c_i=k}(y_i - \mu_k)^2\right)\right].$$

Expand the quadratic term

Define:

$$n_k = \sum_{i=1}^{n} I(c_i = k),$$

the number of observations in cluster k, and

$$S_k = \sum_{i:c_i=k} y_i.$$

Expand:

$$\sum_{i:c_i=k} (y_i - \mu_k)^2 = \sum_{i:c_i=k} y_i^2 - 2\mu_k S_k + n_k \mu_k^2.$$

Substitute back:

$$p(\mu_k \mid y, c) \propto \exp\left[-\frac{1}{2}\left(\frac{\mu_k^2}{\sigma^2} + \sum_{i:c_i=k} y_i^2 - 2\mu_k S_k + n_k \mu_k^2\right)\right].$$

Since $\sum_{i:c_i=k} y_i^2$ does not depend on μ_k, absorb it into the constant:

$$p(\mu_k \mid y, c) \propto \exp\left[-\frac{1}{2}\left(\left(n_k + \frac{1}{\sigma^2}\right)\mu_k^2 - 2S_k\mu_k\right)\right].$$

Complete the square

Let

$$A = n_k + \frac{1}{\sigma^2}.$$

Then:

$$A\mu_k^2 - 2S_k\mu_k = A\left(\mu_k - \frac{S_k}{A}\right)^2 - A\left(\frac{S_k}{A}\right)^2.$$

The second term does not depend on μ_k, so it is absorbed into the constant.

Thus:

$$p(\mu_k \mid y, c) \propto \exp\left[-\frac{A}{2}\left(\mu_k - \frac{S_k}{A}\right)^2\right].$$

This is the kernel of a normal distribution.

Therefore,

$$\mu_k \mid y, c \sim N\left(\frac{S_k}{A}, \frac{1}{A}\right)$$

that is,

$$\mu_k \mid y, c \sim N\left(\frac{\sum_{i:c_i=k} y_i}{n_k + 1/\sigma^2}, \frac{1}{n_k + 1/\sigma^2}\right).$$

Problem 2: Variational inference derivations

Assume a mean-field approximation. Derive a CAVI algorithm for the posterior distribution, $p(\mu, \mathbf{x}|y)$. Show explicit derivations and resulting updates. Implement the algorithm in R. You may refer to Blei, 2017 for details.

Your variational inference R function should

- monitor convergence using the ELBO
- return final estimates of the cluster assignments and means

We consider the Bayesian K-component Gaussian mixture model

$$\mu_k \overset{iid}{\sim} N(0, \sigma^2), \quad k = 1, \dots, K$$

$$c_i \overset{iid}{\sim} \text{Categorical}\left(\frac{1}{K}, \dots, \frac{1}{K}\right), \quad i = 1, \dots, n$$

$$y_i \mid c_i = k, \mu \sim N(\mu_k, 1), \quad i = 1, \dots, n.$$

We want to approximate the posterior distribution

$$p(\mu, c \mid y).$$

Mean-field variational family

Assume the mean-field factorization

$$q(\mu, c) = \prod_{k=1}^{K} q(\mu_k) \prod_{i=1}^{n} q(c_i),$$

with

$$q(\mu_k) = N(m_k, s_k^2)$$

and

$$q(c_i) = \text{Categorical}(\phi_{i1}, \dots, \phi_{iK}), \qquad \sum_{k=1}^{K} \phi_{ik} = 1.$$

Here, ϕ_{ik} is the variational probability that observation i belongs to cluster k.

Under coordinate ascent variational inference, each factor is updated according to

$$\log q_j^*(z_j) = \mathbb{E}_{-j}[\log p(z_j, z_{-j}, y)] + \text{const}.$$

---

1. Derivation of $q(c_i)$

We first derive the optimal variational distribution for c_i.

Using the CAVI formula,

$$\log q^*(c_i) = \mathbb{E}_{q(\mu)\, q(c_{-i})}\big[\log p(\mu, c, y)\big] + \text{const}.$$

The joint distribution is

$$p(\mu, c, y) = p(\mu)\, p(c)\, p(y \mid c, \mu).$$

Among these terms, only $\log p(c_i)$ and $\log p(y_i \mid c_i, \mu)$ depend on c_i. Therefore,

$$\log q^*(c_i = k) = \mathbb{E}\big[\log p(c_i = k) + \log p(y_i \mid c_i = k, \mu)\big] + \text{const}.$$

Because

$$p(c_i = k) = \frac{1}{K},$$

we have

$$\log p(c_i = k) = \log \frac{1}{K},$$

which is constant across k, so it can be absorbed into the normalizing constant.

Also, since

$$y_i \mid c_i = k, \mu \sim N(\mu_k, 1),$$

we have

$$\log p(y_i \mid c_i = k, \mu) = -\frac{1}{2}\log(2\pi) - \frac{1}{2}(y_i - \mu_k)^2.$$

Ignoring terms constant in k,

$$\log q^*(c_i = k) = \mathbb{E}_{q(\mu_k)}\left[-\frac{1}{2}(y_i - \mu_k)^2\right] + \text{const}.$$

Expand the quadratic term:

$$(y_i - \mu_k)^2 = y_i^2 - 2y_i\mu_k + \mu_k^2.$$

Thus,

$$\mathbb{E}[(y_i - \mu_k)^2] = y_i^2 - 2y_i\mathbb{E}[\mu_k] + \mathbb{E}[\mu_k^2].$$

Since $q(\mu_k) = N(m_k, s_k^2)$,

$$\mathbb{E}[\mu_k] = m_k, \qquad \mathbb{E}[\mu_k^2] = m_k^2 + s_k^2.$$

Substituting gives

$$\log q^*(c_i = k) = -\frac{1}{2}\left(y_i^2 - 2y_i m_k + m_k^2 + s_k^2\right) + \text{const}.$$

Dropping the term y_i^2, which does not depend on k, we obtain

$$\log q^*(c_i = k) = y_i m_k - \frac{1}{2}(m_k^2 + s_k^2) + \text{const.}$$

Exponentiating both sides,

$$q(c_i = k) \propto \exp\left(y_i m_k - \frac{1}{2}(m_k^2 + s_k^2)\right).$$

Therefore, the variational responsibilities are

$$\phi_{ik} = \frac{\exp\left(y_i m_k - \frac{1}{2}(m_k^2 + s_k^2)\right)}{\sum_{h=1}^{K} \exp\left(y_i m_h - \frac{1}{2}(m_h^2 + s_h^2)\right)}$$

and

$$q(c_i) = \text{Categorical}(\phi_{i1}, \dots, \phi_{iK}).$$

---

2. Derivation of $q(\mu_k)$

Next, we derive the optimal variational distribution for μ_k.

Using the CAVI formula,

$$\log q^*(\mu_k) = \mathbb{E}_{q(c)\, q(\mu_{-k})}[\log p(\mu, c, y)] + \text{const.}$$

Only the prior term for μ_k and the likelihood terms involving μ_k are relevant. Hence,

$$\log q^*(\mu_k) = \log p(\mu_k) + \sum_{i=1}^{n} \mathbb{E}_{q(c_i)}[\log p(y_i \mid c_i, \mu)] + \text{const.}$$

Since y_i depends on μ_k only when $c_i = k$,

$$\log q^*(\mu_k) = \log p(\mu_k) + \sum_{i=1}^{n} \phi_{ik} \log p(y_i \mid c_i = k, \mu_k) + \text{const.}$$

From the prior,

$$\mu_k \sim N(0, \sigma^2),$$

so

$$\log p(\mu_k) = -\frac{1}{2}\log(2\pi\sigma^2) - \frac{\mu_k^2}{2\sigma^2}.$$

Ignoring constants,

$$\log p(\mu_k) = -\frac{\mu_k^2}{2\sigma^2} + \text{const.}$$

Also, from the likelihood,

$$\log p(y_i \mid c_i = k, \mu_k) = -\frac{1}{2}\log(2\pi) - \frac{1}{2}(y_i - \mu_k)^2.$$

Ignoring constants again,

$$\log p(y_i \mid c_i = k, \mu_k) = -\frac{1}{2}(y_i - \mu_k)^2 + \text{const.}$$

Therefore,

$$\log q^*(\mu_k) = -\frac{\mu_k^2}{2\sigma^2} - \frac{1}{2}\sum_{i=1}^{n} \phi_{ik}(y_i - \mu_k)^2 + \text{const.}$$

Expand the square:

$$(y_i - \mu_k)^2 = y_i^2 - 2y_i\mu_k + \mu_k^2.$$

Substituting,

$$\log q^*(\mu_k) = -\frac{\mu_k^2}{2\sigma^2} - \frac{1}{2}\sum_{i=1}^{n} \phi_{ik}y_i^2 + \sum_{i=1}^{n} \phi_{ik}y_i\mu_k - \frac{1}{2}\sum_{i=1}^{n} \phi_{ik}\mu_k^2 + \text{const.}$$

Collecting the terms involving μ_k,

$$\log q^*(\mu_k) = -\frac{1}{2}\left(\frac{1}{\sigma^2} + \sum_{i=1}^{n} \phi_{ik}\right)\mu_k^2 + \left(\sum_{i=1}^{n} \phi_{ik}y_i\right)\mu_k + \text{const.}$$

This is the kernel of a Gaussian distribution. Therefore,

$$q(\mu_k) = N(m_k, s_k^2),$$

with

$$s_k^2 = \left(\frac{1}{\sigma^2} + \sum_{i=1}^{n} \phi_{ik} \right)^{-1}$$

and

$$m_k = s_k^2 \sum_{i=1}^{n} \phi_{ik} y_i.$$

So the optimal variational factor is

$$q(\mu_k) = N\left(s_k^2 \sum_{i=1}^{n} \phi_{ik} y_i,\ \left(\frac{1}{\sigma^2} + \sum_{i=1}^{n} \phi_{ik} \right)^{-1} \right).$$

---

3. ELBO

The evidence lower bound is

$$\mathcal{L}(q) = \mathbb{E}_q[\log p(\mu, c, y)] - \mathbb{E}_q[\log q(\mu, c)].$$

Using the factorization of the model,

$$\mathcal{L}(q) = \mathbb{E}_q[\log p(\mu)] + \mathbb{E}_q[\log p(c)] + \mathbb{E}_q[\log p(y \mid c, \mu)] - \mathbb{E}_q[\log q(\mu)] - \mathbb{E}_q[\log q(c)].$$

Each term can be computed explicitly.

First,

$$\mathbb{E}_q[\log p(\mu)] = \sum_{k=1}^{K} \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{m_k^2 + s_k^2}{2\sigma^2} \right].$$

Second, since $p(c_i = k) = 1/K$,

$$\mathbb{E}_q[\log p(c)] = \sum_{i=1}^{n} \sum_{k=1}^{K} \phi_{ik} \log \frac{1}{K} = -n \log K.$$

Third,

$$\mathbb{E}_q[\log p(y \mid c, \mu)] = \sum_{i=1}^{n} \sum_{k=1}^{K} \phi_{ik} \left[-\frac{1}{2} \log(2\pi) - \frac{1}{2} \mathbb{E}_q[(y_i - \mu_k)^2] \right].$$

Since

$$\mathbb{E}_q[(y_i - \mu_k)^2] = (y_i - m_k)^2 + s_k^2,$$

this becomes

$$\mathbb{E}_q[\log p(y \mid c, \mu)] = \sum_{i=1}^{n} \sum_{k=1}^{K} \phi_{ik} \left[-\frac{1}{2} \log(2\pi) - \frac{1}{2} ((y_i - m_k)^2 + s_k^2) \right].$$

Fourth, for $q(\mu_k) = N(m_k, s_k^2)$, the entropy term is

$$-\mathbb{E}_q[\log q(\mu)] = \sum_{k=1}^{K} \frac{1}{2} \log(2\pi e s_k^2).$$

Finally, for the categorical factors,

$$-\mathbb{E}_q[\log q(c)] = -\sum_{i=1}^{n} \sum_{k=1}^{K} \phi_{ik} \log \phi_{ik}.$$

Therefore, the full ELBO is

$$\begin{aligned}
\mathcal{L} = & \sum_{k=1}^{K} \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{m_k^2 + s_k^2}{2\sigma^2} \right] - n \log K \\
& + \sum_{i=1}^{n} \sum_{k=1}^{K} \phi_{ik} \left[-\frac{1}{2} \log(2\pi) - \frac{1}{2} ((y_i - m_k)^2 + s_k^2) \right] \\
& + \sum_{k=1}^{K} \frac{1}{2} \log(2\pi e s_k^2) - \sum_{i=1}^{n} \sum_{k=1}^{K} \phi_{ik} \log \phi_{ik}.
\end{aligned}$$

This ELBO can be used to monitor convergence of the CAVI algorithm.

4. Resulting CAVI Algorithm

The coordinate ascent variational inference algorithm alternates between the following two updates.

Update the cluster assignment probabilities

For each $i = 1, \dots, n$,

$$\phi_{ik} = \frac{\exp\left(y_i m_k - \frac{1}{2}(m_k^2 + s_k^2)\right)}{\sum_{h=1}^{K} \exp\left(y_i m_h - \frac{1}{2}(m_h^2 + s_h^2)\right)}.$$

Update the variational distribution of μ_k

For each $k = 1, \dots, K$,

$$s_k^2 = \left(\frac{1}{\sigma^2} + \sum_{i=1}^{n} \phi_{ik}\right)^{-1}$$

$$m_k = s_k^2 \sum_{i=1}^{n} \phi_{ik} y_i.$$

Convergence monitoring

At each iteration, compute the ELBO and stop when the increase in ELBO is smaller than a pre-specified tolerance.

Final estimates

The final estimate of the cluster assignment is

$$\hat{c}_i = \arg\max_k \phi_{ik},$$

and the final estimate of the component means is

$$\hat{\mu}_k = m_k.$$

Problem 3: Fit to data

Implement the Gibb's sampler and variational inference updates in R. Fit to the Old Faithful waiting times dataset used in the EM lecture.

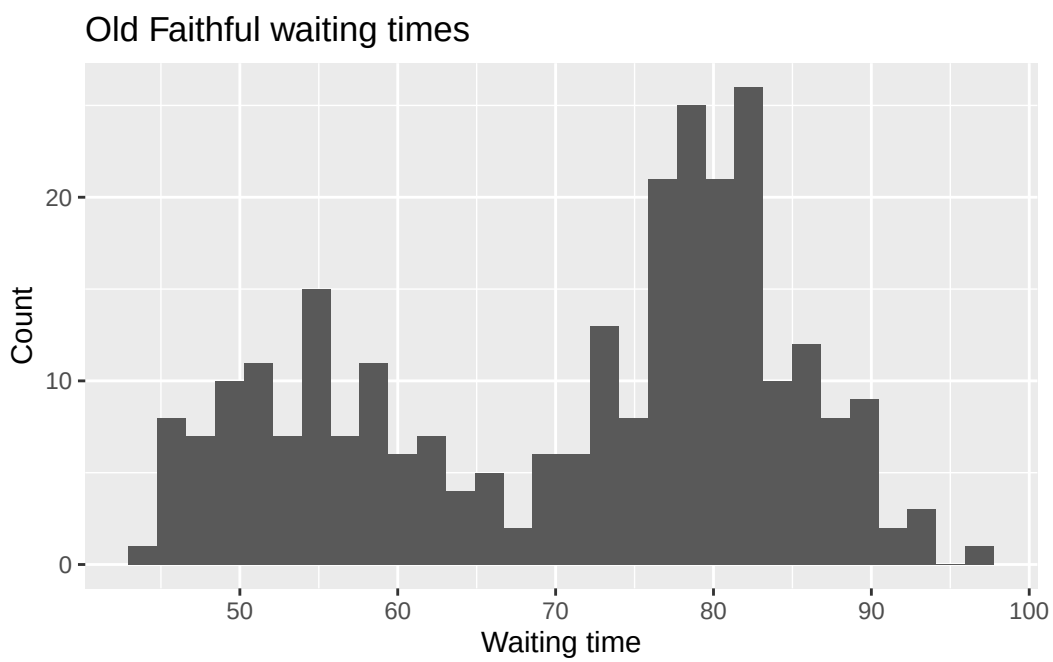
Compare computation times and estimates from variational bayes to mean parameter estimates from the Gibb's sampler. For Gibb's sampler, also do diagnostic plots, fit multiple chains, comment on convergence and effective sample size.

```r
library(tidyverse)
library(coda)
library(knitr)

source("R/helpers.R")
source("R/gibbs_mixture.R")
source("R/vb_mixture.R")
```

```r
library(tidyverse)

data(faithful)
y <- faithful$waiting
n <- length(y)
K <- 2
sigma2 <- 100
summary(y)
```

```
 Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
 43.0    58.0    76.0    70.9    82.0    96.0 
```

```r
ggplot(data.frame(waiting = y), aes(x = waiting)) +
  geom_histogram(bins = 30) +
  labs(
    title = "Old Faithful waiting times",
    x = "Waiting time",
    y = "Count"
  )
```



Gibbs sampler fit

I fit the model using the Gibbs sampler derived in Problem 1. To make posterior summaries interpretable, I addressed label switching by reordering the component means after each iteration so that (_1 < _2), and relabeling cluster assignments accordingly.

```r
set.seed(731)

time_gibbs <- system.time({
  fit_gibbs <- gibbs_mixture(
    y = y,
    K = K,
    sigma2 = sigma2,
```

```r
    n_iter = 10000,
    burn = 2000
  )
})

time_gibbs
```

```
   user  system elapsed
  39.28    0.93   40.53
```

```r
fit_gibbs$mu_post_mean
```

```
[1] 54.74446 80.27924
```

Variational Bayes fit

I also fit the model using coordinate ascent variational inference (CAVI), using the ELBO to monitor convergence.

```r
time_vb <- system.time({
  fit_vb <- vb_mixture(
    y = y,
    K = K,
    sigma2 = sigma2,
    max_iter = 1000,
    tol = 1e-8
  )
})

time_vb
```

```
   user  system elapsed
   0.04    0.00    0.04
```

```r
fit_vb$m
```

```
[1] 54.74453 80.28022
```

Comparison of estimates and computation times

The table below compares posterior mean estimates from the Gibbs sampler with variational mean estimates from VB.

```r
comparison_tbl <- tibble(
  Method = c("Gibbs", "Gibbs", "VB", "VB"),
  Component = c(1, 2, 1, 2),
  Estimate = c(fit_gibbs$mu_post_mean, fit_vb$m)
)

comparison_tbl
```

```
# A tibble: 4 x 3
  Method Component Estimate
  <chr>      <dbl>    <dbl>
1 Gibbs          1     54.7
2 Gibbs          2     80.3
3 VB             1     54.7
4 VB             2     80.3
```

```r
time_tbl <- tibble(
  Method = c("Gibbs", "VB"),
  User_Time = c(time_gibbs["user.self"], time_vb["user.self"]),
  Elapsed_Time = c(time_gibbs["elapsed"], time_vb["elapsed"])
)

time_tbl
```

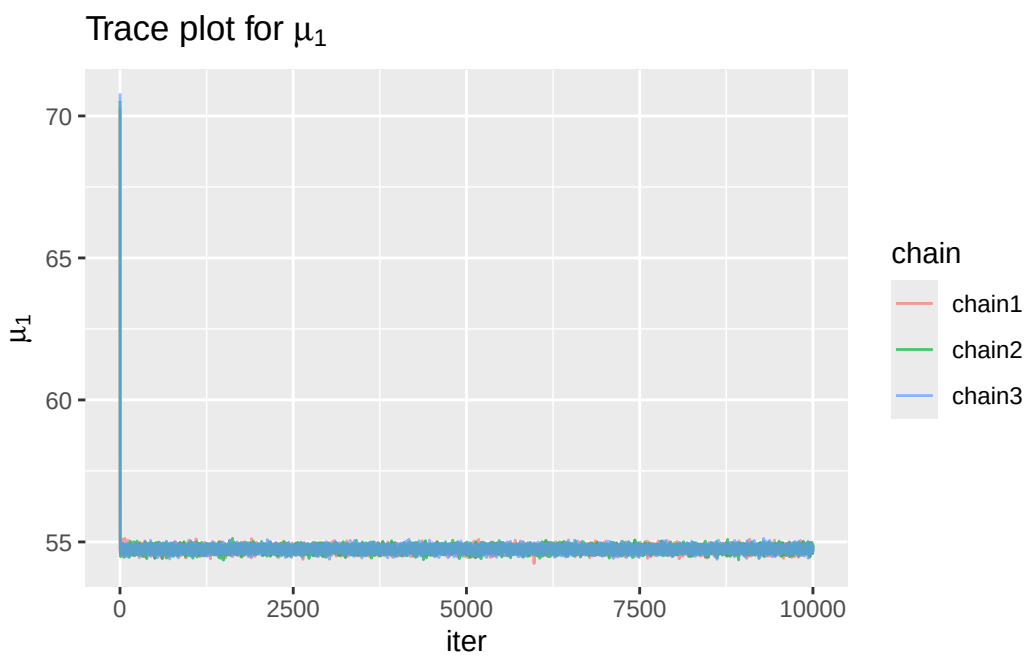
```
# A tibble: 2 x 3
  Method User_Time Elapsed_Time
  <chr>      <dbl>        <dbl>
1 Gibbs    39.3         40.5
2 VB        0.0400       0.0400
```

Gibbs sampler diagnostics

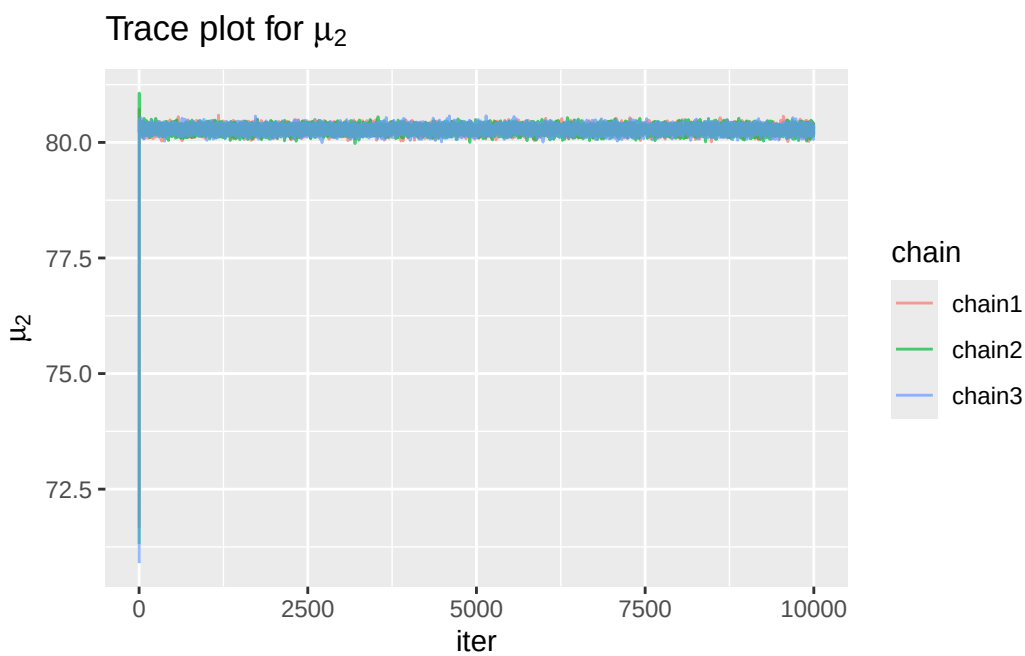
To assess convergence, I ran three Gibbs chains with different initial values.

```r
set.seed(2026)

init1 <- sample(1:K, n, replace = TRUE)
init2 <- ifelse(y < median(y), 1, 2)
init3 <- ifelse(y < mean(y), 2, 1)

fit_gibbs_1 <- gibbs_mixture(y, K = K, sigma2 = sigma2, n_iter = 10000, burn = 2000, init_
fit_gibbs_2 <- gibbs_mixture(y, K = K, sigma2 = sigma2, n_iter = 10000, burn = 2000, init_
fit_gibbs_3 <- gibbs_mixture(y, K = K, sigma2 = sigma2, n_iter = 10000, burn = 2000, init_
```

```r
trace_df <- bind_rows(
  tibble(iter = 1:10000, mu1 = fit_gibbs_1$mu_draws[,1], mu2 = fit_gibbs_1$mu_draws[,2], c
  tibble(iter = 1:10000, mu1 = fit_gibbs_2$mu_draws[,1], mu2 = fit_gibbs_2$mu_draws[,2], c
  tibble(iter = 1:10000, mu1 = fit_gibbs_3$mu_draws[,1], mu2 = fit_gibbs_3$mu_draws[,2], c
)

ggplot(trace_df, aes(x = iter, y = mu1, color = chain)) +
  geom_line(alpha = 0.7) +
  labs(title = expression("Trace plot for " * mu[1]), y = expression(mu[1]))
```



```r
ggplot(trace_df, aes(x = iter, y = mu2, color = chain)) +
  geom_line(alpha = 0.7) +
  labs(title = expression("Trace plot for " * mu[2]), y = expression(mu[2]))
```

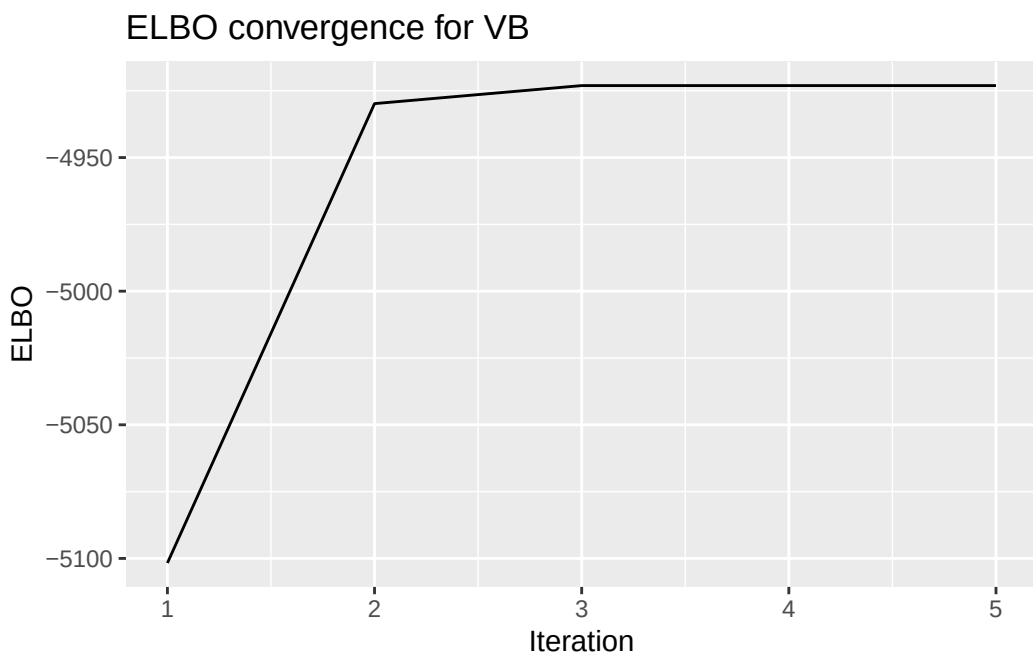



```r
chain_mu1 <- mcmc.list(
  mcmc(fit_gibbs_1$mu_draws[fit_gibbs_1$keep, 1, drop = FALSE]),
  mcmc(fit_gibbs_2$mu_draws[fit_gibbs_2$keep, 1, drop = FALSE]),
  mcmc(fit_gibbs_3$mu_draws[fit_gibbs_3$keep, 1, drop = FALSE])
)

chain_mu2 <- mcmc.list(
  mcmc(fit_gibbs_1$mu_draws[fit_gibbs_1$keep, 2, drop = FALSE]),
  mcmc(fit_gibbs_2$mu_draws[fit_gibbs_2$keep, 2, drop = FALSE]),
  mcmc(fit_gibbs_3$mu_draws[fit_gibbs_3$keep, 2, drop = FALSE])
)

ess_tbl <- tibble(
  Parameter = c("mu1", "mu2"),
  ESS = c(effectiveSize(chain_mu1), effectiveSize(chain_mu2)),
  Rhat = c(
    gelman.diag(chain_mu1)$psrf[1,1],
```

```r
      gelman.diag(chain_mu2)$psrf[1,1]
    )
  )

  ess_tbl
```

```
# A tibble: 2 x 3
  Parameter    ESS  Rhat
  <chr>      <dbl> <dbl>
1 mu1       23242.  1.00
2 mu2       24162.  1.00
```

```r
  elbo_df <- tibble(
    iter = seq_along(fit_vb$elbo),
    elbo = fit_vb$elbo
  )

  ggplot(elbo_df, aes(x = iter, y = elbo)) +
    geom_line() +
    labs(
      title = "ELBO convergence for VB",
      x = "Iteration",
      y = "ELBO"
    )
```



Discussion

Both the Gibbs sampler and variational Bayes identified a clear two-cluster structure in the Old Faithful waiting time data. The estimated component means from the two approaches were similar, suggesting that the variational approximation captured the main posterior location reasonably well.

Variational Bayes was substantially faster than Gibbs sampling because it uses deterministic coordinate updates instead of posterior sampling. However, Gibbs sampling provides full posterior draws and therefore allows uncertainty quantification and convergence diagnostics. The trace plots from three chains suggested reasonable mixing after burn-in, and the effective sample sizes were adequate for the component means. Overall, VB provided a fast approximation, while Gibbs sampling gave richer inferential output.

Problem 4: Perform a simulation study on the cluster

In our simulation, we want to

- Generate data from a 4-part Gaussian mixture model with true means $\mu = \{0, 5, 10, 20\}$ and unit variance.
  - Evaluate estimation of μ through bias, coverage, and computation time
  - We will compare the Gibbs sampler and Variational Bayes algorithms defined above

- Evaluate these properties at sample sizes $n = \{100, 1000, 10000\}$
  - Use Gibbs chains of length 10000 and discard the first 2000 samples as burn in
  - Use nsim = 500

We will execute this simulation study on the cluster. For full credit, make sure to implement the following:

Workflow: * Use structured scripts and subfolders following guidance from the cluster computing project organization lecture * Instead of parallelizing your simulation scenarios (as in HW1), each simulation scenario should be assigned a different JOBID on the cluster.

Presenting results:

Create plots with *Monte Carlo standard error bars* to summarize the following:

- Bias of $\hat{\mu}$
- Coverage of $\hat{\mu}$

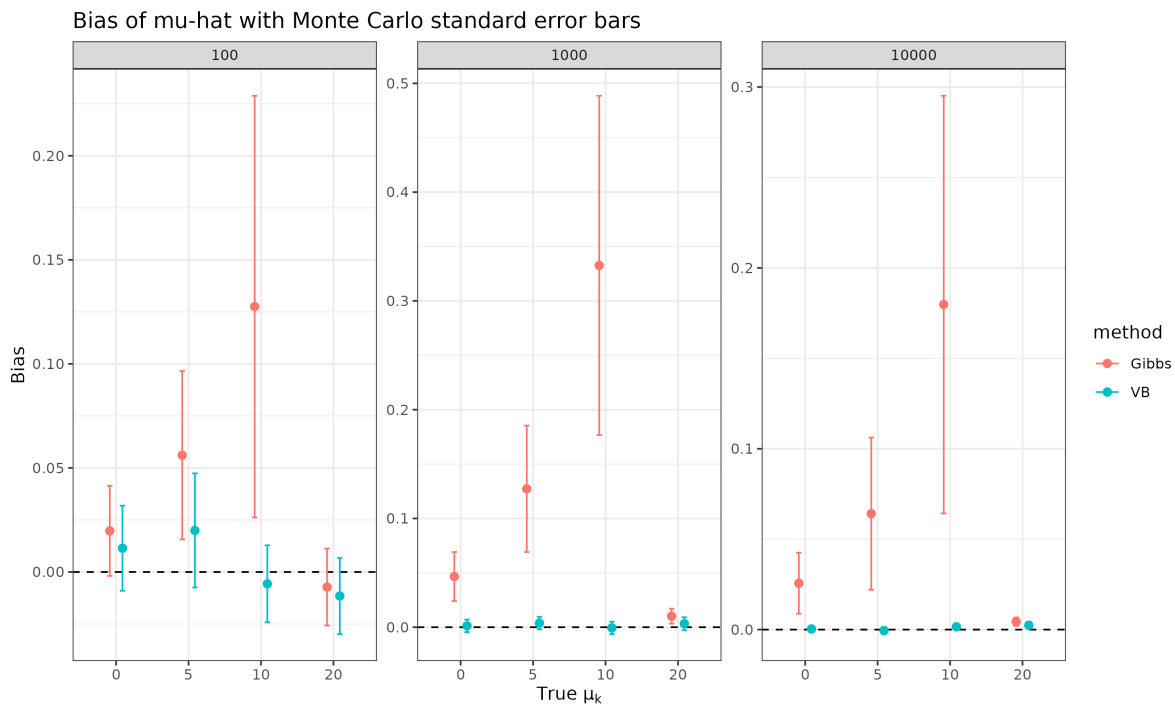
Write 1-2 paragraphs summarizing these results.

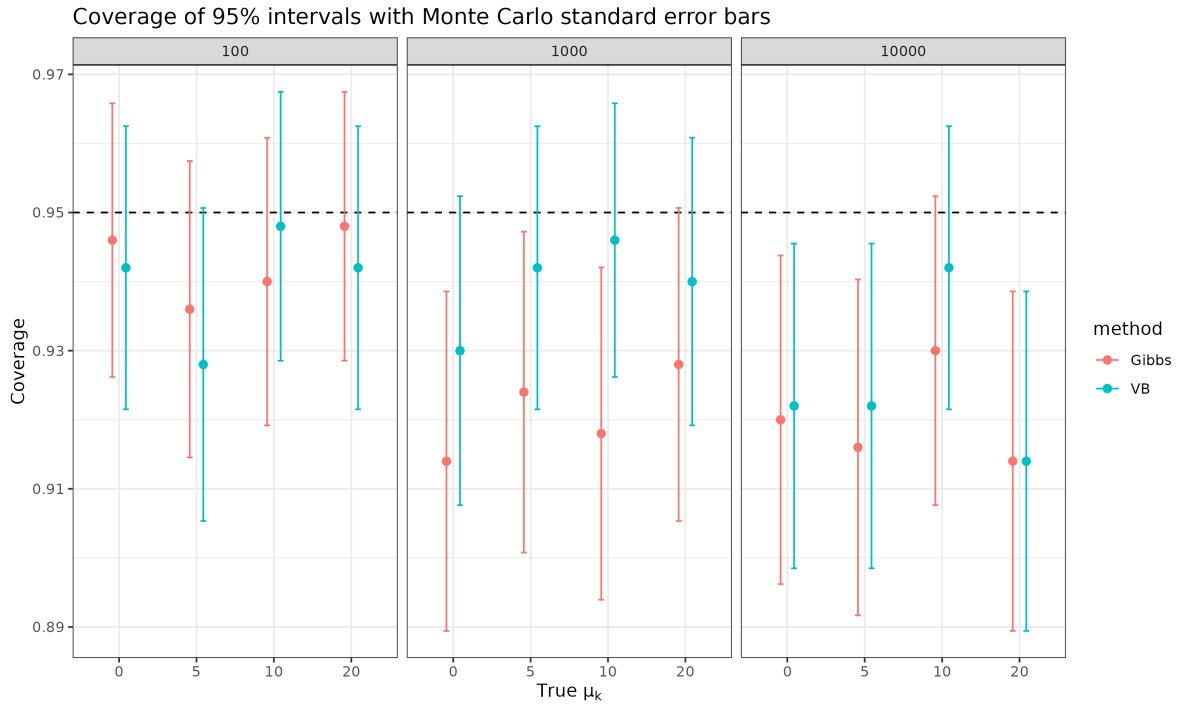
```r
library(tidyverse)

summary_metrics <- readr::read_csv("results/summary/summary_metrics.csv", show_col_types =
time_summary <- readr::read_csv("results/summary/time_summary.csv", show_col_types = FALSE

summary_metrics
```

```
# A tibble: 24 x 12
   method     n component mu_true  nsim mean_mu_hat     bias bias_mcse coverage
   <chr>  <dbl>     <dbl>   <dbl> <dbl>       <dbl>    <dbl>     <dbl>    <dbl>
 1 Gibbs    100         1       0   500      0.0197  0.0197    0.0110     0.946
 2 Gibbs    100         2       5   500      5.06    0.0561    0.0206     0.936
 3 Gibbs    100         3      10   500     10.1     0.128     0.0517     0.94
 4 Gibbs    100         4      20   500     20.0    -0.00727   0.00944    0.948
 5 Gibbs   1000         1       0   500      0.0466  0.0466    0.0115     0.914
 6 Gibbs   1000         2       5   500      5.13    0.127     0.0297     0.924
 7 Gibbs   1000         3      10   500     10.3     0.333     0.0796     0.918
 8 Gibbs   1000         4      20   500     20.0     0.0102    0.00351    0.928
 9 Gibbs  10000         1       0   500      0.0256  0.0256    0.00859    0.92
10 Gibbs  10000         2       5   500      5.06    0.0641    0.0215     0.916
# i 14 more rows
# i 3 more variables: coverage_mcse <dbl>, mean_elapsed_time <dbl>,
#   median_elapsed_time <dbl>
```

```r
time_summary
```

```
# A tibble: 6 x 8
  method     n  nsim mean_elapsed_time sd_elapsed_time median_elapsed_time
  <chr>  <dbl> <dbl>             <dbl>           <dbl>               <dbl>
1 Gibbs    100   500            22.5             5.15                23.9
2 Gibbs   1000   500           179.             38.4                156.
3 Gibbs  10000   500          2259.            539.                2423.
4 VB       100   500             0.0629          0.0156              0.0650
5 VB      1000   500             0.210           0.0671              0.210
6 VB     10000   500             1.76            0.423               1.90
# i 2 more variables: min_elapsed_time <dbl>, max_elapsed_time <dbl>
```





Across all sample sizes, variational Bayes (VB) produced highly accurate point estimates with negligible bias for all mixture components, and the bias decreased further as the sample size increased. In contrast, Gibbs sampling showed noticeably larger bias for some components, especially for the component with true mean 10, where the bias remained positive across all sample sizes. Despite this, both methods generally estimated the extreme components ((=0) and (=20)) reasonably well. Overall, VB provided more stable point estimation performance across simulation settings.

In terms of uncertainty quantification, both methods achieved coverage rates reasonably close to the nominal 95% level, although coverage tended to be slightly below 95%, especially for larger sample sizes. Gibbs sampling had somewhat more variable coverage across components, while VB showed slightly more consistent performance. Computationally, VB was substantially faster than Gibbs sampling in every scenario. For example, at (n=10000), the average runtime for VB was about 1.76 seconds, compared with over 2259 seconds for Gibbs sampling. These results suggest that VB provides a much more computationally efficient alternative while maintaining competitive estimation accuracy and interval coverage in this setting.